



第三章 语法分析



语法分析是编译过程的核心。

语法分析是对高级程序设计语言的句子结构进行分析，它的任务是识别输入的单词序列是否符合语言的语法规则，如果符合通常就生成语法树。

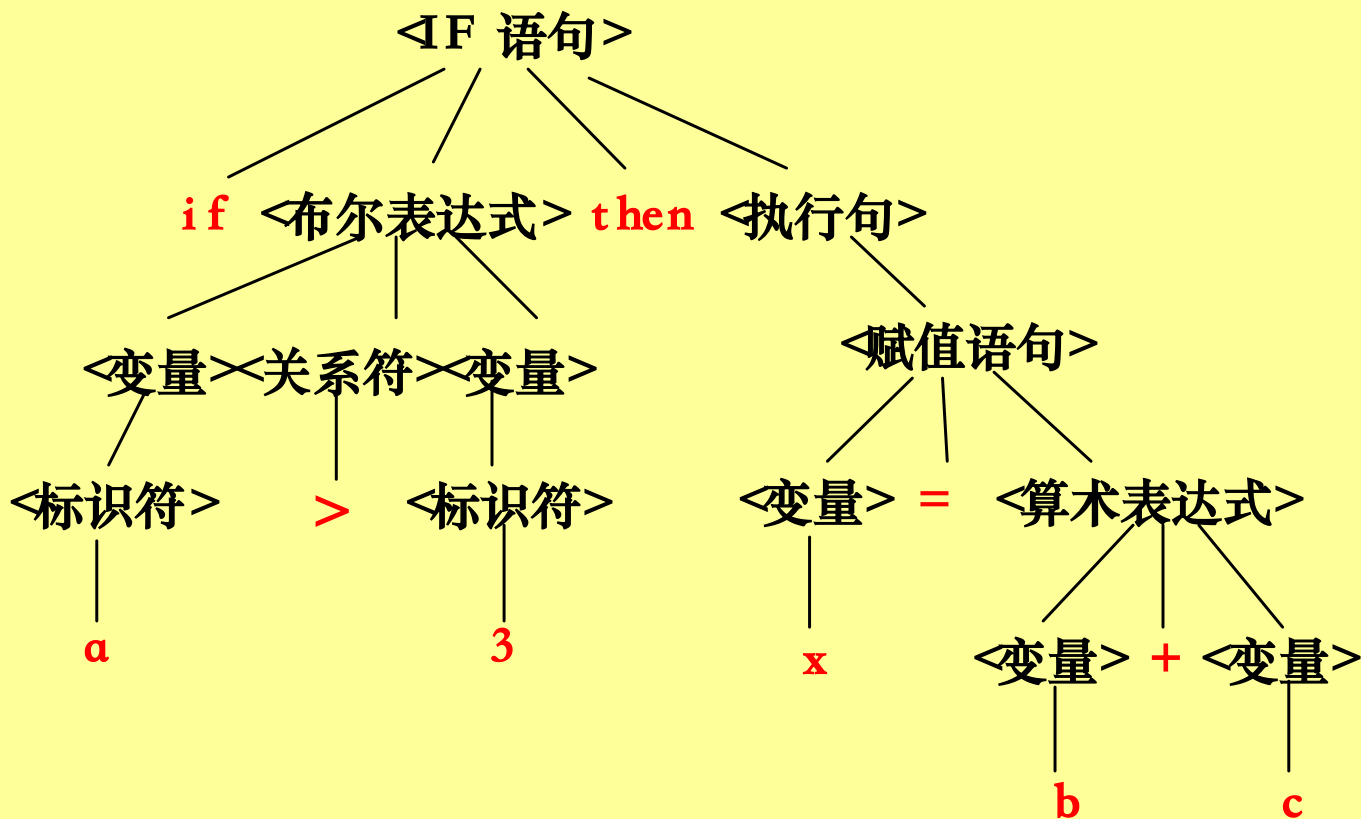
根据生成语法树的方法，可将语法分析分成自上而下方法和自下而上方法。

输出

if

源文件

```
/*this is test*/  
if a>3 then  
    x=b+c;
```



如何知道这些正确的单词是否能构成正确的语法单位
(表达式、语句或程序) ?

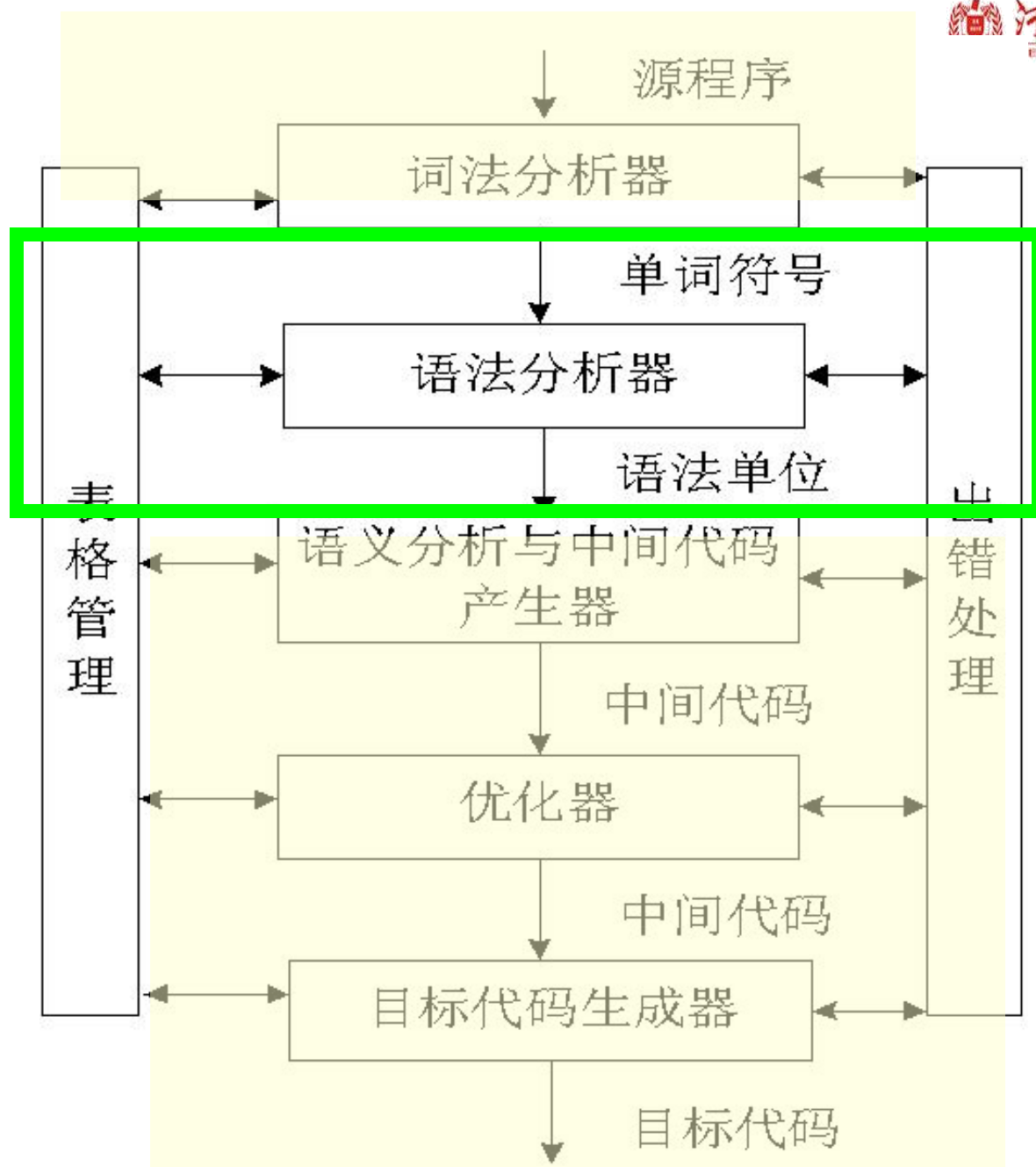
输出内容是什么?

语法分析的任务：

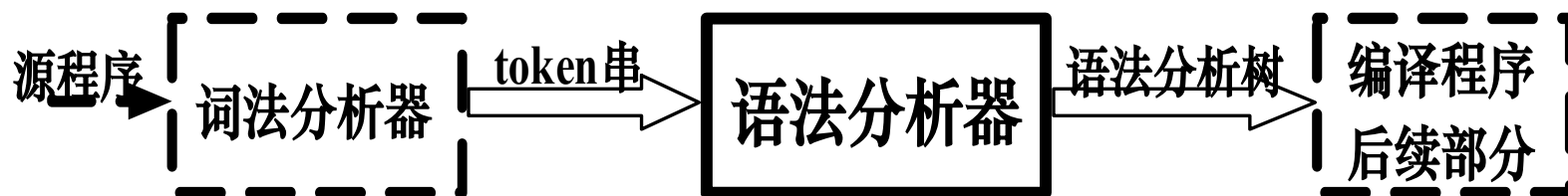
分析并判定输入单词序列是否符合语法规则（即是否是给定文法的一个句子）。需要对程序设计语言的语法构成规则进行形式化描述。

高级语言的语法构成规则适合用上下文无关文法来描述，上下文无关文法是语法分析的基础。

3.1 语法分析概述



在词法分析识别出正确的单词符号串的基础上，分析并判定这些输入符号串是否符合语法规则（即是否是给定文法的一个句子）。



- 语法分析器的功能

- 按照语言的语法规则，识别单词能否构成一个语法正确的程序。规则用产生式定义。

- 语法分析器的输入

- Token序列：词法分析的输出，是各个单词都正确的源程序的变换形式，是一个有限序列

- 语法分析器的输出

- 分析树：

- 错误处理信息：错误定位、继续编译

• 问题

- 对给定的一个输入符号串，如何判定它是不是一个句子？
- 每一种程序设计语言都具有描述其语法结构的规则，程序设计语言的这种构成规则使用上下文无关文法来描述。
- 按照文法手工实现或自动生成一个有效地语法分析程序，用来判断输入的符号串在语法上是否正确。
- 方法
- 根据文法的产生式规则，看能否建立与输入串匹配的语法树。
- 有两种方法

语法分析方法(常用的方法)

➤ 自上而下分析法:

从文法的开始符号出发，使用最左推导，尽可能使用各种产生式，推导出与输入串匹配的句子，从而建立语法树。

➤ 自下而上分析法:

从输入符号串开始，逐步进行归约(最右推导的逆过程)，直至归约到文法的开始符号，从而建立语法树。

➤ 具体分类:

自上而下分析	递归下降分析
	预测分析(LL)
自下而上分析	算符优先分析
	LR分析

3.2 上下文无关文法



- 程序设计语言的语法结构的形式化描述称为**文法**。
是一种工具，用于严格定义句子的结构；用有穷的规则刻划无穷的集合
- 文法是被用来精确而无歧义地描述语言的句子的构成方式
- 文法描述语言的时候不考虑语言的含义

引 例

例1：有如下规则

<句子> → <主语><谓语>

<主语> → <代词>|<名词>

<代词> → 我|你|他

<名词> → 大学生|工人|王明|英语

<谓语> → <动词><直接宾语>

<动词> → 是|学习

<直接宾语> → <代词>|<名词>

• 现要求根据如上规则得出句子：

• 我是大学生

<句子>

=><主语><谓语>

=><代词><谓语>

=>我<谓语>

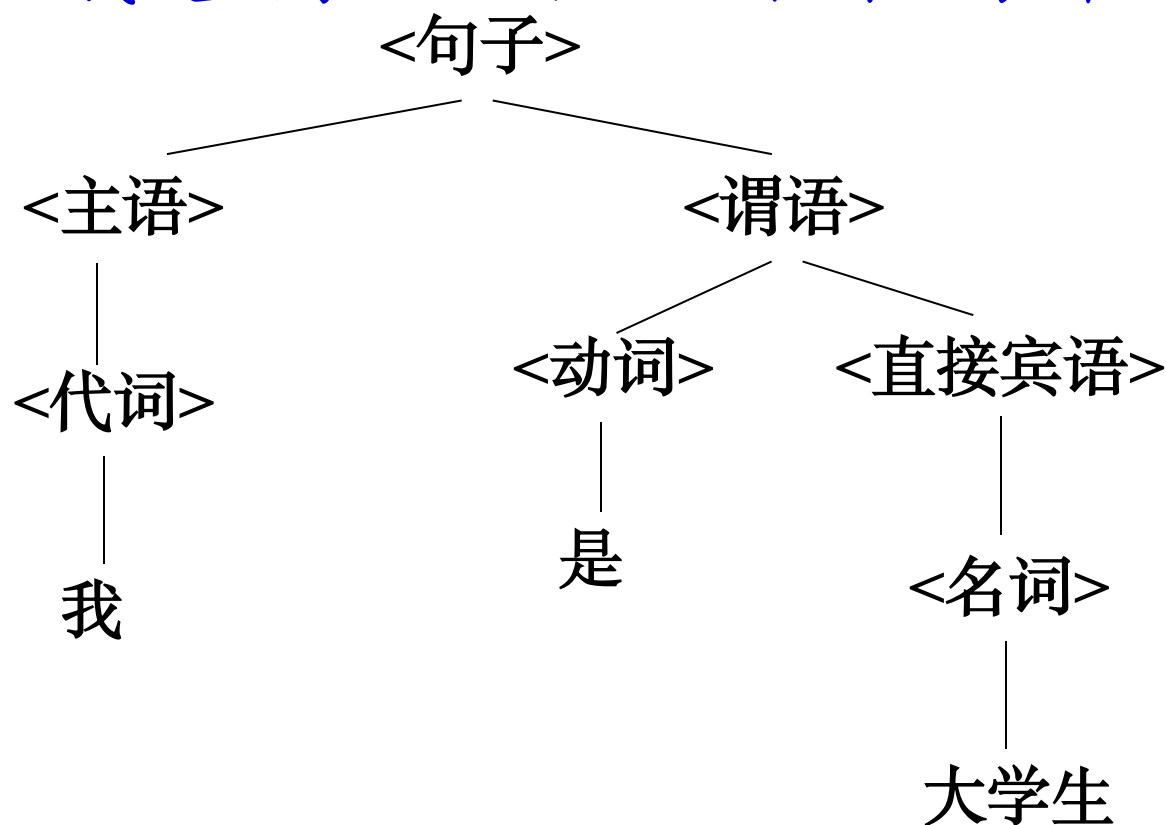
=>我<动词><直接宾语>

=>我是<直接宾语>

=>我是<名词>

=>我是大学生

句子“我是大学生”可以如下图示分析



- 在有规则的情况下，每一次用上述规则的右边去替换左边，得到“我是大学生”是符合上述规则的句子

从例子中抽象出概念

- **终结符号**：是组成该语言的最基本的符号，是不可再分的基本符号，记为 V_T
- **非终结符号**：规则中用尖括号括起来的符号，表示一些语法成分，可以推导出其他的语法成分，记为 V_N
- **开始符号**：规则中的一个特殊的非终结符号，语言中的句
- **产生式**：
成式，说
方式，形
部， β 称
非终结符
则

<句子> $\rightarrow$ <主语><谓语>

<主语> $\rightarrow$ <代词>|<名词>

<代词> $\rightarrow$ 我|你|他

<名词> $\rightarrow$ 大学生|工人|王明|英语

<谓语> $\rightarrow$ <动词><直接宾语>

<动词> $\rightarrow$ 是|学习

<直接宾语> $\rightarrow$ <代词>|<名词>

3.2.1 文法的定义

- 一个文法 G 抽象地表示为四元组

$$G = (V_n, V_t, P, S)$$

其中 V_n 表示非终结符号

V_t 表示终结符号, $V_n \cup V_t$ (字母表),

$$V_n \cap V_t = \phi$$

S 是开始符号,

P 是产生式, 形如: $\alpha \rightarrow \beta$ ($\alpha \in V^+$ 且至少含有一个非终结符号, $\beta \in V^*$)

上例中： $G = (V_n, V_t, P, \langle \text{句子} \rangle)$

$V_n = (\langle \text{句子} \rangle, \langle \text{主语} \rangle, \langle \text{谓语} \rangle, \langle \text{代词} \rangle, \langle \text{动词} \rangle, \langle \text{名词} \rangle, \langle \text{直接宾语} \rangle)$

$V_t = (\text{我}, \text{你}, \text{他}, \text{是}, \text{学习}, \text{大学生}, \text{工人}, \text{王明}, \text{英语})$

$P =$

$\langle \text{句子} \rangle \rightarrow \langle \text{主语} \rangle \langle \text{谓语} \rangle$

$\langle \text{主语} \rangle \rightarrow \langle \text{代词} \rangle | \langle \text{名词} \rangle$

$\langle \text{代词} \rangle \rightarrow \text{我} | \text{你} | \text{他}$

$\langle \text{名词} \rangle \rightarrow \text{大学生} | \text{工人} | \text{王明} | \text{英语}$

$\langle \text{谓语} \rangle \rightarrow \langle \text{动词} \rangle \langle \text{直接宾语} \rangle$

$\langle \text{动词} \rangle \rightarrow \text{是} | \text{学习}$

$\langle \text{直接宾语} \rangle \rightarrow \langle \text{代词} \rangle | \langle \text{名词} \rangle$

例：某语言中标识符定义的文法 $G = (V_n, V_t, P, S)$

其中： $V_n = \{\text{标识符}, \text{字母}, \text{数字}\}$

$V_t = \{a, b, c, \dots, y, z, 0, 1, \dots, 9\}$

$S = \langle \text{标识符} \rangle$

$P = \{ \langle \text{标识符} \rangle \rightarrow \langle \text{字母} \rangle$

$\langle \text{标识符} \rangle \rightarrow \langle \text{标识符} \rangle \langle \text{字母} \rangle$

$\langle \text{标识符} \rangle \rightarrow \langle \text{标识符} \rangle \langle \text{数字} \rangle$

$\langle \text{字母} \rangle \rightarrow a$

$\langle \text{字母} \rangle \rightarrow b$

.....

$\langle \text{字母} \rangle \rightarrow z$

$\langle \text{数字} \rangle \rightarrow 0$

.....

$\langle \text{数字} \rangle \rightarrow 9 \}$

- 在书写产生式时一般有下列约定：
- 第一条产生式的左部是开始符号，在产生式中，用大写字母（或尖括号括起来的）表示非终结符，用小写字母表示终结符，如果S是文法G的开始符号，也可以将文法写成G[S]
- 有时不用将文法的四元组显示给出，只需给出产生式
- 下面是一个文法的等价写法：

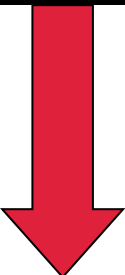
(1) $G = (\{S, A\}, \{a, b\}, P, S)$
P: $S \rightarrow aAb$
 $A \rightarrow ab$
 $A \rightarrow aAb$

(3) $G[S]: S \rightarrow aAb$
 $A \rightarrow ab | aAb$

(2) $G: S \rightarrow aAb$
 $A \rightarrow ab$
 $A \rightarrow aAb$

算术表达式的文法定义

- 变量是表达式
- 表达式 + 表达式是表达式
- 表达式 * 表达式是表达式
- (表达式) 是 表达式


$$\begin{aligned}E &\rightarrow E + E \\E &\rightarrow E * E \\E &\rightarrow (E) \\E &\rightarrow i\end{aligned}$$

$$E \rightarrow E + E \mid E * E \mid (E) \mid i$$

3.2.2 推导

语言用文法定义，那么文法如何产生语言呢？

- **推导**：连续使用产生式右部去替换左部某个非终结符的过程，得到的连续序列称为一个推导。

例： $S \rightarrow 01, 0S0 \Rightarrow 0010$

一步推导又称直接推导(用符号 $\Rightarrow$ 表示)，就是用某条规则的右部去替换该规则的左部。

3.2.2

推导

- 直接推导：如果 $\alpha \rightarrow \beta$ 是文法 G 的产生式， γ 和 δ 是 V^* 中的任意符号，若有符号串 v, w 满足：
 $v = \gamma \alpha \delta$, $w = \gamma \beta \delta$ ，则说 v 直接产生 w ，(w 是 v 的直接推导)记作： $v \Rightarrow w$



如果存在

$v \Rightarrow w_0 \Rightarrow w_1 \Rightarrow w_2 \dots \Rightarrow w_n = w (n > 0), \Rightarrow$ 则称 v

推导出 w (长度为 n) , 记作 $v \Rightarrow w$

(至少一步)

*

$\Rightarrow$

• 若有 $v \Rightarrow w$ 或 $v = w$, 则记作 $v \Rightarrow w$ (0步或若干步)

• 例 : $G = (\{E\}, \{i, +, *, (,)\}, P, E)$

$P: E \rightarrow E+E \mid E * E \mid (E) \mid i$

表达式(i)和(i+i)*i的推导:

$$E \Rightarrow (E) \Rightarrow (i)$$

$$E \Rightarrow E * E \Rightarrow (E) * E \Rightarrow (E + E) * E \Rightarrow (i + E) * E \\ \Rightarrow (i + i) * E \Rightarrow (i + i) * i$$

	$\xRightarrow{*}$	E	0
步推	$\xRightarrow{+}$		
	$\xRightarrow{*}$	$(i + i) * i$	6步推
导	$\xRightarrow{+}$		
	E	$(i + i) * i$	6步推
导	E	(E)	1步推

3.2.3

文法产生的语言

• 例 : $G = (\{E\}, \{i, +, *, (,)\}, P, E)$

$P: E \rightarrow E+E \mid E * E \mid (E) \mid i$

$E \Rightarrow (E) \Rightarrow (i)$

$E \Rightarrow E * E \Rightarrow (E) * E \Rightarrow (E + E) * E \Rightarrow (i + E) * E \Rightarrow (i + i) * E \Rightarrow (i + i) * i$

在推导的每一步都要做两个选择：选择被替换的非终结符和选择非终结符的候选式。如何选择？

3.2.3

文法产生的语言

- 第一个问题，如何选择被替换的非终结符？

- **最左推导**：在整个推导过程中，任何一步推导 $\alpha \Rightarrow \beta$ 都是对 α 中最左边的非终结符进行替换。
- **最右推导**：在整个推导过程中，任何一步推导 $\alpha \Rightarrow \beta$ 都是对 α 中最右边的非终结符进行替换

3.2.3

文法产生的语言

例: $G = (\{E\}, \{i, +, *, (,)\}, P, E)$

$P: E \rightarrow E + E \mid E * E \mid (E) \mid i$

则 $(i+i)*i$ 的最左和最右推导过程为:

最左推导过程:

$$E \Rightarrow E * E \Rightarrow (E) * E \Rightarrow (E + E) * E \Rightarrow (i + E) * E \Rightarrow (i + i) * E \Rightarrow (i + i) * i$$

最右推导过程:

$$E \Rightarrow E * E \Rightarrow E * i \Rightarrow (E) * i \Rightarrow (E + E) * i \Rightarrow (E + i) * i \Rightarrow (i + i) * i$$

2 句型、句子和语言

句型：若 S 是文法 G 的开始符号，从开始符号 S 出发推导出的符号串称为文法 G 的一个句型。

句子：若 x 是文法 G 的一个句型，且 $x \in V_t^*$ 即 x 仅由终结符号组成，则称 x 为 $G(S)$ 的句子

• 文法 G 所产生的语言定义为：

$L(G) = \{x \mid \overset{*}{S} \Rightarrow x, \text{ 其中 } S \text{ 为文法的开始符号, } x \in V_t^*\}$ 。即：
一个文法 G 可以推导出的所有句子构成的集合，称为 G 产生的语言。

3.2.3 文法产生的语言

- 例：考虑文法G：
它定义了什么语言。

$$S \rightarrow bA$$

$$A \rightarrow aA|a$$

推导过程： $S \Rightarrow bA \Rightarrow ba$

$$S \Rightarrow bA \Rightarrow baA \Rightarrow baa$$

.....

$$S \Rightarrow bA \Rightarrow baA \Rightarrow \dots \Rightarrow ba\dots a$$

归纳得： $L(G1) = \{ba^n \mid n \geq 1\}$

3.2.3

文法产生的语言

- 考虑文法G: $S \rightarrow aSb$

$$S \rightarrow ab$$

定义了什么语言

$$L(G) = \{a^n b^n \mid n \geq 1\}$$

• 练习：文法 $G = (\{A, B, S\}, \{a, b, c\}, P, S)$

$S \rightarrow Ac|aB$

$A \rightarrow ab$

$B \rightarrow bc$

写出 $L(G)$ 的全部元素

$L(G) = \{abc\}$

$S \rightarrow abc$

结合上面的练习，思考下面的问题：
一个语言的文法唯一吗？

- 文法等价：

若文法G1和文法G2所产生的语言相同，即 $L(G1) = L(G2)$ ，则称文法G1和文法G2等价。

例：G1[A]: $A \rightarrow 0R$ $A \rightarrow 01$ $R \rightarrow A1$

G2[S]: $S \rightarrow 0S1$ $S \rightarrow 01$

程序设计语言文法描述

1 表达式

表达式是构成程序设计语言中语句的最基本的成分。

- 表达式由运算对象和运算符组成。
- 分为逻辑表达式、关系表达式和算术表达式
- 运算符之间的优先关系和结合性规定了表达式的计算次序。





算术表达式

$\langle \text{算术表达式} \rangle \rightarrow \langle \text{常量} \rangle | \langle \text{变量} \rangle | (\langle \text{算术表达式} \rangle)$
 $| \langle \text{算术表达式} \rangle \langle \text{算术运算符} \rangle \langle \text{算术表达式} \rangle$

$\langle \text{算术表达式} \rangle \rightarrow \langle \text{算术表达式} \rangle + \langle \text{项} \rangle$
 $| \langle \text{算术表达式} \rangle - \langle \text{项} \rangle | \langle \text{项} \rangle$
 $\langle \text{项} \rangle \rightarrow \langle \text{项} \rangle * \langle \text{因子} \rangle | \langle \text{项} \rangle / \langle \text{因子} \rangle | \langle \text{因子} \rangle$
 $\langle \text{因子} \rangle \rightarrow (\langle \text{算术表达式} \rangle) | \langle \text{常量} \rangle | \langle \text{变量} \rangle$
 $\langle \text{变量} \rangle \rightarrow \langle \text{标识符} \rangle$
 $\langle \text{常量} \rangle \rightarrow \langle \text{整型常量} \rangle | \langle \text{实型常量} \rangle$

逻辑表达式

<逻辑表达式> → <布尔常量>

<逻辑表达式> → <布尔变量>

<逻辑表达式> → (<关系表达式>)

<逻辑表达式> → not <逻辑表达式>

<逻辑表达式> → <逻辑表达式> and <逻辑表达式>

<逻辑表达式> → <逻辑表达式> or <逻辑表达式>



关系表达式

$\langle \text{关系表达式} \rangle \rightarrow \langle \text{算术表达式} \rangle \langle \text{关系运算符} \rangle \langle \text{算术表达式} \rangle$

$\langle \text{关系运算符} \rangle \rightarrow <|>|<>|<=|>=|==$





2 语句

程序由各种语句构成

- 语句
 - 说明性语句
 - 可执行语句
- 说明性语句旨在定义各种不同数据类型的变量或运算，不需要由编译程序生成目标代码，主要用来告诉编译程序一些实体的属性，供编译程序生成目标代码时使用。
- 可执行语句旨在描述程序的动作，需要由编译程序生成目标代码来实现它的语义。

说明语句

<说明语句> → <常量说明>|<变量说明>

<常量说明> → const <标识符> = <常数>

<变量说明> → var <变量表>:<类型>

<变量表> → <变量>|<变量表>,<变量>

<变量> → <标识符>

<类型> → integer|real|char|boolean



执行语句

<执行语句> → <读语句>|<写语句>| <赋值语句>
>|<控制语句>|<复合语句>

<读语句> → read (<变量>)

<写语句> → write (<变量>)

<赋值语句> → <变量>=<表达式>

<表达式> → <算术表达式>|<逻辑表达式>| <关系
表达式>

<控制语句> → <if语句>|<while语句>|<for语句>
>|<dowhile语句>

<复合语句> → begin <语句表> end

.....

3 程序

- 程序是语言中最大的语法单位。设计时应考虑程序的总体结构，各子程序之间的关系，如何构成一个程序等。

<程序> → program <标识符
>; <程序体>

<程序体> → <常量说明><变
量说明><复合句>.



符合SAMPLE语言规范的源程序

```
/*this is a sample program writing in Sample language*/  
program example1;  
/*used for illustrating compiling process*/  
var  
    a,b,c:integer;  
    x:char;  
begin  
    a=2;  
    read(b);  
    read(c);  
    x=' stu' ;  
    if a>0 and (x<>' ' ) then b=a+b*c;  
    else  
        for b= 1 to 100 do c=c+b;  
        while c>=1 do  
            begin  
                b=b*c;  
                c=c-1;  
            end;  
        do a=10; while a>b;  
    end.
```

小结

- 文法
- 推导 最左推导 最右推导
- 句型 句子 语言



例: $G = (\{E\}, \{i, +, *, (,)\}, P, E)$

$P: E \rightarrow E + E \mid E * E \mid (E) \mid i$

表达式 $(i+i)*i$ 的推导过程:

$$\begin{aligned} (1) \quad E &\Rightarrow E + E \Rightarrow E + E + E \Rightarrow i + E + E \\ &\Rightarrow i + i + E \Rightarrow i + i + i \end{aligned}$$

$$\begin{aligned} (2) \quad E &\Rightarrow E + E \Rightarrow i + E \Rightarrow i + E + E \Rightarrow i + i + E \\ &\Rightarrow i + i + i \end{aligned}$$

推导句子的过程不唯一，原因有两个，一是一个非终结符有多个候选式，二是推导的顺序不同。推导过程有没有直观表示方法吗？